já sei um pouco de C#, mas quero aprender na prática desenvolvimento de API's .NET/C# usando Entity, Swagger, testes (acho q seria XUnit certo?)... quero aprender como funciona a organização, a estrutura, etc... vou desenvolver com o CODEX, pq ai ele vai desenvolvendo, me guiando e me mostrando o que eu devo aprender aos poucos... qual tema de API vc me recomenda? quero fazer algo simples, pra aprender mesmo, nada muito complexo, somente alguns cruds e talvez um login com jwt... me recomende alguns temas por favor

Boa, isso aqui é exatamente o tipo de projeto que te leva de "sei um pouco" → "consigo entregar backend real" 🚀

Vou te passar ideias de API **simples, mas com densidade técnica boa** (CRUD + JWT + boas práticas), sem firula — foco em aprendizado com ROI alto.

🔥 Melhores temas pra sua API (.NET + Entity + Swagger + XUnit)

1. 📚 API de Gerenciamento de Tarefas (TODO PRO+)

Clássico, mas dá pra elevar o nível

Features:

- CRUD de tarefas
- Status (pendente, em progresso, concluída)
- Datas (deadline, createdAt)
- Relacionamento com usuário
- Filtro por status/data

Extra:

- Login com JWT
- Usuário só vê as próprias tasks
- Paginação + ordenação

👉 Por que é bom:

- Te ensina **90% do que você precisa no dia a dia real**
- Simples, mas dá pra sofisticar fácil

---

2. 💸 API de Controle Financeiro (tipo Mobills simplificado)

Esse aqui é MUITO valorizado em entrevista

Features:

- CRUD de transações (entrada/saída)
- Categorias (alimentação, lazer, etc)
- Soma automática (saldo total)
- Filtro por período

Extra:

- JWT (cada usuário com seus dados)
- Relatório mensal (agrupamento)

👉 Por que é forte:

- Trabalha **queries mais reais**
- Aprende bem LINQ + agregações

---

3. 🛒 API de E-commerce Simplificado

Mais próximo de sistema real

Features:

- Produtos (CRUD)
- Categorias
- Carrinho
- Pedido

Extra:

- JWT
- Relacionamentos (User → Orders → Items)

👉 Valor:

- Aprende **relacionamentos mais complexos**
- Já entra no mindset de microserviços

---

4. 🎮 API de Biblioteca de Jogos (essa é sua cara 👀)

Gamificado = mais motivação

Features:

- CRUD de jogos
- Status (jogando, zerado, backlog)
- Nota (rating)
- Plataforma (PS, PC, etc)

Extra:

- Usuário com JWT
- Ranking de jogos favoritos

👉 Por que recomendo forte:

- Você vai curtir fazer → mantém consistência
- Dá pra expandir depois (reviews, etc)

---

5. 🧠 API de Sistema de Notas/Estudos

Simples + estruturado

Features:

- CRUD de notas
- Tags
- Busca por texto

Extra:

- JWT
- Organização por categorias

👉 Valor:

- Aprende **busca + filtros**
- Estrutura limpa

🧱 Stack que você deve usar (pra aprender certo)

Sem inventar moda:

- ASP.NET Core Web API
- Entity Framework Core
- SQL Server ou SQLite (pra começar rápido)

- Swagger (Swashbuckle)
- XUnit + FluentAssertions
- JWT Authentication

🧠 Estrutura que você DEVE praticar (isso é o ouro)

Foca nisso aqui:

```
/Controllers
/Services
/Repositories
/Entities
/DTOs
/Data (DbContext)
/Tests
```

👉 E aplica:

- Separation of Concerns
- Dependency Injection
- DTO != Entity
- Repository Pattern (mesmo que simples)

🧪 Testes (XUnit mesmo, boa escolha)

Começa com:

- Teste de Service (regra de negócio)
- Mock de repository (Moq)

Exemplo de foco:

- "não pode criar task sem título"
- "não pode acessar task de outro usuário"

🧠 Minha recomendação direta (sem enrolar)

Se quer eficiência máxima:

👉 **Escolhe um desses dois:**

1. Controle Financeiro 💸
2. Biblioteca de Jogos 🎮

Esses dois têm o melhor custo-benefício pra aprender **backend real**

🚀 Roadmap de execução (bem pragmático)

1. CRUD básico (sem auth)
2. Entity + migrations
3. DTOs
4. Services + validações
5. Swagger redondinho
6. JWT auth
7. Relacionamento entre entidades
8. Testes com XUnit

Se quiser, no próximo passo eu te monto:

👉 **um escopo completo tipo "RFC" do projeto escolhido**
(com entidades, endpoints, estrutura de pastas e até prompts pro Codex)

Aí você já começa nível produção, não tutorialzinho 👍

sobre os exemplos que vc sugeriu:
gerenciamento de tarefas (TODO), controle financeiro, e-comerce e biblioteca de jogos, todas achei mais complexo e eu pretendo fazer depois com uma tecnologia que domino mais...
como seria essa API de Sistema de Notas/Estudos? detalhe um pouco mais ela por favor... quais seriam as entidades e relacionamentos?

Pensou por um segundo

Boa. Essa de **Sistema de Notas/Estudos** é bem mais tranquila pra aprender backend sem já cair num mini monolito corporativo com trauma de brinde.

Ela é boa porque você pratica quase tudo que importa em API sem precisar lidar com regra de negócio muito cabeluda.

Ideia da API

Seria uma API onde o usuário pode:

- criar anotações
- organizar por categorias
- adicionar tags
- marcar como favorita, arquivada ou fixada
- buscar notas por texto
- filtrar por categoria ou tag

Dá até pra pensar como um **mini Notion / Evernote / Obsidian backend simplificado**.

O que você aprenderia com ela

Com esse projeto, você consegue praticar:

- CRUD
- relacionamentos 1:N e N:N
- autenticação com JWT
- Entity Framework
- migrations
- DTOs
- paginação e filtros
- busca
- testes de service
- organização em camadas

Ou seja: pra estudo, ela é humilde por fora e fortinha por dentro.

Entidades principais

1. User

Representa o usuário do sistema.

Campos exemplo:

- Id
- Name
- Email
- PasswordHash
- CreatedAt

Relacionamento

- **1 usuário tem muitas notas**
- **1 usuário tem muitas categorias**

---

2. Note

É a anotação em si.

Campos exemplo:

- Id
- Title
- Content
- IsFavorite
- IsArchived
- IsPinned
- CreatedAt
- UpdatedAt
- UserId
- CategoryId

Relacionamento

- **muitas notas pertencem a 1 usuário**
- **muitas notas podem pertencer a 1 categoria**
- **muitas notas podem ter muitas tags**

---

3. Category

Serve pra organizar as notas.

Campos exemplo:

- Id
- Name
- Color
- UserId

Relacionamento

- **1 categoria pertence a 1 usuário**
- **1 categoria pode ter muitas notas**

Exemplo:

- Trabalho
- Estudos
- Ideias
- Backlog

---

4. Tag

Marcadores mais flexíveis.

Campos exemplo:

- Id
- Name
- UserId

Relacionamento

- **1 tag pertence a 1 usuário**
- **1 tag pode estar em muitas notas**
- **1 nota pode ter muitas tags**

Exemplo:

- angular
- backend
- csharp
- urgente

---

5. NoteTag

Tabela de junção do relacionamento N:N entre Note e Tag.

Campos:

- NoteId
- TagId

Relacionamento

- uma nota pode ter várias tags
- uma tag pode estar em várias notas

Relacionamentos resumidos

Fica assim:

- **User 1:N Note**
- **User 1:N Category**
- **User 1:N Tag**
- **Category 1:N Note**
- **Note N:N Tag** via **NoteTag**

Visualmente:

```
User
 ├── Notes
 ├── Categories
 └── Tags

Category
 └── Notes

Note
 └── Tags (via NoteTag)
```

Estrutura mínima do projeto

Eu faria algo assim:

```
/src
  /YourProject.Api
    /Controllers
    /DTOs
    /Mappings
    /Middlewares
    /Configurations

  /YourProject.Application
    /Services
    /Interfaces
    /Validators

  /YourProject.Domain
    /Entities
    /Enums

  /YourProject.Infrastructure
    /Data
    /Repositories
    /Migrations
```

Mas como você quer aprender sem complicar demais, pode começar com uma versão mais enxuta:

```
/Controllers
/Services
/Repositories
/Entities
/DTOs
/Data
/Tests
```

Essa já tá ótima pra começo.

Fluxo de uso da API

Auth

- `POST /auth/register`
- `POST /auth/login`

Retorna JWT.

Notes

- `GET /notes`
- `GET /notes/{id}`
- `POST /notes`
- `PUT /notes/{id}`
- `DELETE /notes/{id}`

Filtros legais:

- `GET /notes?search=jwt`
- `GET /notes?categoryId=1`
- `GET /notes?tag=backend`
- `GET /notes?isFavorite=true`
- `GET /notes?isArchived=false`

Categories

- `GET /categories`

- `POST /categories`
- `PUT /categories/{id}`
- `DELETE /categories/{id}`

Tags

- `GET /tags`
- `POST /tags`
- `DELETE /tags/{id}`

Exemplo de payload

Criar nota

```
{
  "title": "Estudar JWT",
  "content": "Aprender claims, expiration e authorization",
  "categoryId": 2,
  "tagIds": [1, 3],
  "isFavorite": true
}
```

Regras de negócio simples e boas pra praticar

Aqui mora o suco.

User

- e-mail não pode repetir
- senha precisa ser hasheada
- usuário só acessa os próprios dados

Category

- não permitir categoria duplicada por usuário com mesmo nome

Tag

- não permitir tag duplicada por usuário com mesmo nome

Note

- título obrigatório
- nota deve pertencer ao usuário autenticado
- categoria, se enviada, também deve pertencer ao mesmo usuário
- tags associadas também devem ser do mesmo usuário

Isso já te ensina a fazer validação séria, não só CRUD de tutorial de indiano no YouTube.

Como usar JWT nesse projeto

Depois do login, o token carrega pelo menos:

- UserId
- Email

Aí nos endpoints protegidos você usa o UserId do token pra:

- buscar só as notas do cara
- impedir acesso a recurso de outro usuário

Exemplo:
o usuário 1 não pode editar nota do usuário 2.

Esse tipo de regra é excelente pra aprender backend de verdade.

O que testar com XUnit

Service de Auth

- deve registrar usuário com sucesso
- não deve registrar email duplicado
- deve falhar com senha inválida
- deve gerar token ao logar

Service de Note

- deve criar nota
- não deve criar nota sem título
- não deve associar categoria de outro usuário
- não deve associar tag de outro usuário
- deve retornar só notas do usuário logado
- deve atualizar somente nota do próprio usuário

Service de Category/Tag

- não deve duplicar nome
- deve remover corretamente

Versão 1 ideal pra você

Pra não virar bagunça, eu começaria assim:

V1

- User
- Auth com JWT
- Note
- Category
- Tag
- NoteTag
- CRUD completo
- busca por texto
- filtro por categoria/tag
- testes de service

Isso já tá perfeito.

Versão 2 depois

Quando quiser evoluir:

- paginação
- soft delete
- ordenação por data
- notas fixadas no topo
- anexos
- histórico de edição
- compartilhamento de notas

Mas isso é depois. Agora é arroz, feijão e músculo de backend.

Minha opinião sincera

Esse projeto é uma ótima escolha porque ele é:

- simples de entender
- útil pra aprender arquitetura

- fácil de evoluir
- pequeno o suficiente pra terminar
- rico o bastante pra praticar backend real

Ele tem menos “peso mental” que e-commerce e financeiro, mas ainda te ensina coisa que vale pra qualquer API.

Se quiser, eu posso montar agora a **estrutura completa desse projeto**, com:

- entidades em C#
- endpoints
- ordem de implementação
- e um plano passo a passo pro Codex ir construindo com você.

Beleza, vamo lá! gostei das susas ideias mas tenho alguns pontos:
-Qual vai ser o nome do projeto? algo como StudyNotesAPI?
-Crie um .md detalhando tudo isso, com um passo a passo muito bem estruturado em varias etapas de implementação, para que o codex possa implementar item a item, crud a crud...
-Gostei do que vc falou como "Estrutura mínima do projeto", não precisa fazer a versão simplificada/enxuta, pode estruturar as pastas na estrutura que vc falou antes mesmo (com /YourProject.Api, /YourProject.Application, /YourProject.Domain, /YourProject.Infrastructure e tests, pq afinal, é assim mesmo que são estruturados as API's normalmente em C#/.NET, certo?)
-Pode fazer tudo o que vc mencionou no V1 e também pode incluir a paginação e a ordenação que vc mencionou na versão 2... mas a ordenação não vai ser apenas por data, mas por todo os possíveis dados aplicaveis da entidade
-Estabeleça todos os scripts necessários por favor pra rodar o projeto, rodar os testes, o coverage dos testes (que deve ser de 100% em todos os arquivos possíveis, exceto aquele não aplicaveis, tipo de configuração), rodar a doc com o swagger, tudo detalhado no readme do projeto... e crie também o que seria o .env desse projeto,

por favor! o banco de dados pode ser criado localmente mesmo e use mysql, não sql server
-Enfim! Me fale o nome do projeto e me passe esse md estruturado por favor

Vou fechar isso num blueprint de projeto mesmo: nome, estrutura em camadas, roadmap por etapas, endpoints, regras, scripts, cobertura e um `.env.example` pra você plugar no Codex sem sofrimento.

Pensou por 3m 27s

Fechou: o nome que eu recomendo é **StudyNotesApi**.
Tá clean, profissional e sem cara de projeto de tutorial com cheiro de CRUD genérico.

Preparei o `.md` estruturado com:

- nome e objetivo do projeto
- arquitetura em camadas
- entidades e relacionamentos
- regras de negócio
- endpoints
- paginação e ordenação
- roadmap em etapas pro Codex implementar item por item
- comandos pra rodar API, migrations, testes e coverage
- `.env.example`
- orientação de README

Arquivo aqui:

Baixar o blueprint em Markdown

Minha opinião reta: **StudyNotesApi** ficou melhor que algo como `StudyNotesAPI` só por padronização visual de nome de projeto .NET mesmo. Fica mais bonito em namespace, solution e repo.

Se quiser, no próximo passo eu já posso te entregar a versão 2 desse material: um **README.md pronto** e também a **árvore inicial real da solution com os comandos** `dotnet new`, `dotnet sln add`, `dotnet add reference` **e os pacotes NuGet exatos**.